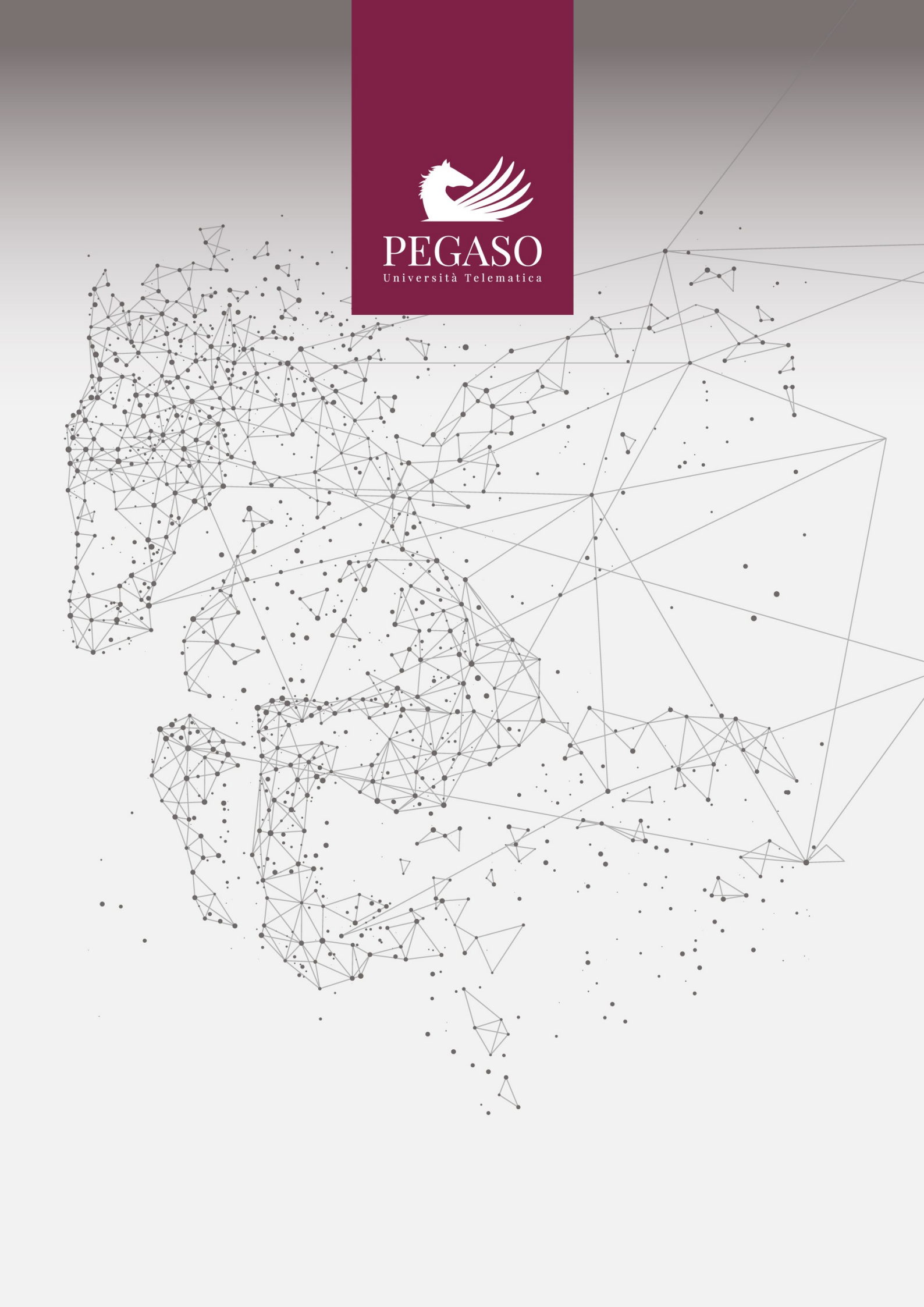

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. MVC....3
2. MIDDLEWARE....5
3. PASSPORT....7
4. PASSPORT & OAUTH....13
BIBLIOGRAFIA....16
SITOGRAFIA....17

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. MVC

MVC (Model-View-Controller) è un pattern architetturale utilizzato per separare le responsabilità del codice in tre parti distinte: il **Model**, la **View** e il **Controller**.

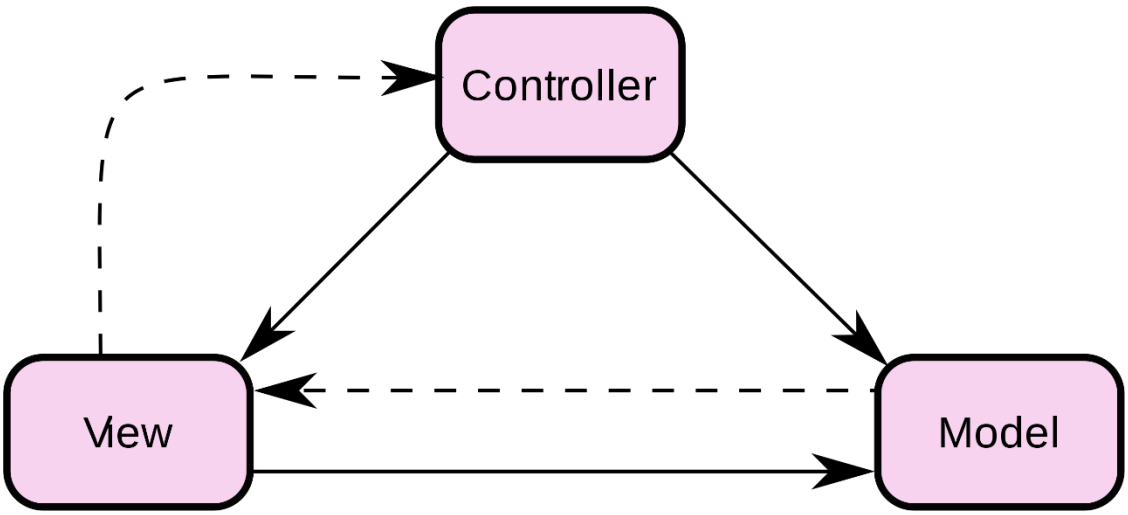


***Figura 1** – MVC*

- Il **Model** rappresenta i dati dell'applicazione e definisce le operazioni per accedere e manipolare questi dati. Il Model non conosce nulla della View o del Controller, ma fornisce solo un'interfaccia per accedere ai dati.
- La **View** rappresenta l'interfaccia utente dell'applicazione e visualizza i dati forniti dal Model. La View non ha alcuna conoscenza del Model o del Controller, ma fornisce solo un'interfaccia utente per interagire con l'applicazione.
- Il **Controller** si occupa di coordinare l'interazione tra il Model e la View. Il Controller riceve le richieste dell'utente dalla View, accede ai dati del Model per elaborare la richiesta e aggiorna la View con i dati elaborati.

L'utilizzo del pattern MVC ha diversi vantaggi, tra cui una maggiore separazione delle responsabilità del codice, una maggiore modularità, una maggiore flessibilità e una maggiore facilità di manutenzione e testabilità.

In Node.js, il pattern MVC può essere implementato utilizzando un framework web come Express.js. Ad esempio, si può utilizzare Express per gestire le richieste dell'utente (Controller), definire rotte e middleware per l'accesso ai dati dell'applicazione (Model) e utilizzare template engine come Handlebars o EJS per creare le View.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Vediamo nel dettaglio i singoli componenti del pattern MVC e come possono essere implementati in Node.js.

- Il **Model** rappresenta i dati dell'applicazione e definisce le operazioni per accedere e manipolare questi dati. In Node.js, il Model può essere implementato utilizzando una libreria di accesso ai database come Sequelize, Mongoose o Knex.js. Ad esempio, Sequelize è un ORM (Object-Relational Mapping) che permette di definire i modelli dei dati dell'applicazione e di interagire con il database utilizzando oggetti JavaScript.
- La **View** rappresenta l'interfaccia utente dell'applicazione e visualizza i dati forniti dal Model. In Node.js, la View può essere implementata utilizzando un template engine come Handlebars, EJS o Pug. Ad esempio, Handlebars permette di creare template HTML dinamici utilizzando una sintassi simile a quella di Mustache.
- Il **Controller** si occupa di coordinare l'interazione tra il Model e la View. In Node.js, il Controller può essere implementato utilizzando le funzionalità di routing di Express.js. Ad esempio, si può utilizzare Express per definire le rotte dell'applicazione e le funzioni che gestiscono le richieste HTTP. In questo modo, il Controller accede ai dati del Model per elaborare la richiesta e aggiorna la View con i dati elaborati.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Middleware

Un middleware è una funzione che può accedere all'oggetto richiesta (request) e all'oggetto risposta (response) dell'applicazione, e che può anche modificare il flusso di esecuzione delle richieste HTTP che passano attraverso l'applicazione.

Un middleware può essere utilizzato per molte cose diverse, come ad esempio:

- **Validazione dei dati**: ad esempio, verificare che una richiesta POST contenga tutti i campi richiesti.
- **Autenticazione**: ad esempio, verificare che l'utente sia autenticato prima di consentirgli di accedere ad alcune parti dell'applicazione.
- **Compressione delle risposte**: ad esempio, comprimere le risposte in formato JSON per ridurre la quantità di dati trasmessi.
- **Logging**: ad esempio, registrare tutte le richieste HTTP e le relative risposte per fini di monitoraggio e debug.
- **Eseguire operazioni comuni**: ad esempio, inserire un header comune in ogni risposta HTTP.

I middleware in Node.js sono eseguiti in cascata, ovvero uno dopo l'altro. L'ordine in cui i middleware sono definiti è importante, perché determina l'ordine in cui vengono eseguiti. Inoltre, è possibile utilizzare middleware a livello di applicazione o a livello di singola route:

- i middleware a livello di applicazione vengono eseguiti per tutte le richieste HTTP che passano attraverso l'applicazione;
- i middleware a livello di route vengono eseguiti solo per le richieste HTTP che corrispondono a quella specifica route.

Ogni middleware riceve tre parametri:

- l'oggetto richiesta (request);
- l'oggetto risposta (response);
- la funzione next.

La funzione next() viene utilizzata per passare il controllo al prossimo middleware nella catena di gestione delle richieste. In questo modo, è possibile creare una serie di funzioni che vengono eseguite in sequenza, ognuna delle quali può effettuare elaborazioni specifiche sulla richiesta e sulla risposta: ad esempio, si può creare un middleware per la gestione degli errori, che viene eseguito in caso di errori durante la gestione della richiesta, oppure un middleware per la gestione della sessione dell'utente.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

L'utilizzo dei middleware consente di separare le varie funzioni della gestione delle richieste in blocchi autonomi, rendendo il codice più modulare, riutilizzabile e più facile da mantenere.

Un esempio di middleware potrebbe essere quello che verifica se l'utente che sta effettuando una determinata richiesta è autenticato o meno.

Supponiamo di avere un'applicazione Express che gestisce le richieste HTTP in ingresso. Possiamo creare un middleware che controlla se l'utente ha effettuato l'accesso, e se sì, passa la richiesta al successivo middleware o all'handler della richiesta finale; in caso contrario, può reindirizzare l'utente alla pagina di login.

Ecco un esempio di come potrebbe essere implementato un middleware per il controllo dell'autenticazione:

```
function isAuthenticated(req, res, next) {
  if (req.session && req.session.user) {
    return next();
  } else {
    return res.redirect('/login');
  }
}
```

In questo caso, il middleware controlla se l'utente ha una sessione attiva e se c'è un oggetto user all'interno della sessione: se l'utente è autenticato, il middleware passa la richiesta al successivo middleware o all'handler della richiesta finale chiamando next(); in caso contrario, reindirizza l'utente alla pagina di login.

Il middleware può essere utilizzato in questo modo all'interno delle definizioni delle rotte:

```
app.get('/dashboard', isAuthenticated, function(req, res) {
  // qui si arriva solo se l'utente è autenticato
  res.render('dashboard', { user: req.session.user });
});
```

In questo caso, il middleware isAuthenticated viene utilizzato come secondo parametro della funzione app.get(), in modo che venga eseguito prima dell'handler che renderizza la pagina di dashboard; se l'utente non è autenticato, il middleware reindirizza l'utente alla pagina di login e l'handler non viene eseguito.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Passport

Relativamente al tema middleware, Passport è un middleware per autenticazione in Node.js utilizzato per autenticare le richieste dei client verso le applicazioni web. Passport consente di definire diversi meccanismi di autenticazione, come ad esempio l'autenticazione tramite username e password, l'autenticazione tramite Facebook, Twitter, Google, ecc.

Passport utilizza il concetto di "strategie" per gestire l'autenticazione: una strategia di autenticazione è un modulo di Node.js che definisce come verrà gestita l'autenticazione per un particolare provider di autenticazione (ad esempio, un provider di autenticazione locale o un provider di autenticazione OAuth).

Quando un utente tenta di autenticarsi, Passport utilizza la strategia corrispondente per gestire la richiesta di autenticazione: la strategia viene eseguita attraverso una chiamata a una funzione di Passport, che viene solitamente denominata "authenticate"; questa funzione prende in input le informazioni di autenticazione dell'utente (ad esempio, nome utente e password) e restituisce un callback che viene chiamato alla fine del processo di autenticazione.

La strategia di autenticazione può accettare o rifiutare l'autenticazione in base alle informazioni di autenticazione fornite dall'utente: se l'autenticazione viene accettata, la strategia restituisce un oggetto "user" che contiene le informazioni sull'utente autenticato. Queste informazioni possono essere utilizzate per creare una sessione utente o per altre operazioni di gestione utente.

Passport supporta diverse strategie di autenticazione, tra cui:

- **Local authentication strategy**: utilizzata per autenticare gli utenti utilizzando un sistema di autenticazione locale, ad esempio un database di utenti; questa strategia richiede di fornire un nome utente e una password per verificare l'identità dell'utente.
- **Token-based authentication strategy**: utilizzata per autenticare gli utenti utilizzando token, ad esempio JSON Web Token (JWT); questa strategia richiede di fornire un token valido per verificare l'identità dell'utente.
- **OAuth authentication strategy**: utilizzata per autenticare gli utenti utilizzando OAuth, un protocollo di autorizzazione che consente agli utenti di autorizzare l'accesso alle loro risorse da parte di terze parti; questa strategia richiede di fornire un token OAuth valido per verificare l'identità dell'utente.
- **OpenID authentication strategy**: utilizzata per autenticare gli utenti utilizzando OpenID, un protocollo di autenticazione che consente agli utenti di autenticarsi utilizzando le loro

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

credenziali di accesso di un provider di autenticazione; questa strategia richiede di fornire un token OpenID valido per verificare l'identità dell'utente.

Esistono anche altre strategie di autenticazione, come ad esempio la strategia di autenticazione a due fattori e la strategia di autenticazione biometrica, che possono essere utilizzate per implementare meccanismi di autenticazione più sicuri.

Per gestire le informazioni utente, Passport utilizza il concetto di **serializzazione**: la serializzazione è il processo di conversione dei dati utente in un formato che può essere memorizzato in modo persistente (ad esempio, su un database o su un cookie). Quando un utente effettua l'accesso all'applicazione, i dati utente vengono serializzati e salvati in un'area di memoria persistente. La serializzazione può essere utilizzata anche per memorizzare altre informazioni importanti, come ad esempio il ruolo dell'utente o le autorizzazioni di accesso.

La **deserializzazione**, d'altra parte, è il processo inverso di conversione dei dati utente memorizzati in formato persistente in un oggetto utente. Quando un utente effettua una richiesta successiva alla sessione, i dati utente vengono deserializzati e utilizzati per creare un oggetto utente valido. Questo oggetto può quindi essere utilizzato per gestire l'autenticazione e l'autorizzazione dell'utente.

Per gestire la serializzazione e la deserializzazione, Passport utilizza due funzioni speciali: **serializeUser** e **deserializeUser**.

- La funzione **serializeUser** viene chiamata quando l'utente effettua l'accesso all'applicazione: questa funzione prende in input l'oggetto utente e restituisce un identificatore univoco per l'utente, che viene utilizzato per serializzare i dati utente in modo persistente. Ad esempio, se l'oggetto utente ha un ID utente univoco, la funzione serializeUser potrebbe restituire semplicemente l'ID utente.
- La funzione **deserializeUser** viene chiamata quando l'utente effettua una richiesta successiva alla sessione: questa funzione prende in input l'identificatore dell'utente serializzato e restituisce l'oggetto utente corrispondente. Ad esempio, se l'identificatore dell'utente è l'ID utente univoco, la funzione deserializeUser potrebbe utilizzare l'ID utente per cercare l'oggetto utente corrispondente nel database.

Analizziamo i vari passi necessari per configurare Passport nell'applicazione Node.js.

Occorre installare Passport e le eventuali strategie di autenticazione necessarie tramite npm: se si desidera utilizzare la strategia di autenticazione locale, è possibile installare Passport e la strategia locale utilizzando il comando:

```
npm install passport passport-local
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Successivamente occorre importare Passport nell'applicazione Node.js e inizializzarlo: questo può essere fatto nel file principale dell'applicazione, ad esempio app.js o server.js

```
const passport = require('passport');
const app = express();
app.use(passport.initialize());
```

A questo punto è necessario configurare le strategie di autenticazione: questo può essere fatto all'interno del file principale dell'applicazione o in un file separato, ad esempio auth.js; di seguito è riportato un esempio di configurazione della strategia di autenticazione locale:

```
const LocalStrategy = require('passport-local').Strategy;
const User = require('./models/user');

passport.use(new LocalStrategy(
  function(username, password, done) {
    User.findOne({ username: username }, function (err, user) {
      if (err) { return done(err); }
      if (!user) { return done(null, false); }
      if (!user.verifyPassword(password)) { return done(null, false); }
      return done(null, user);
    });
  }
));
```

In questo esempio, si sta utilizzando la strategia di autenticazione locale e si sta cercando un utente nel database in base al nome utente: se l'utente viene trovato, viene verificata la password e viene restituito un oggetto utente valido.

Al fine di consentire la serializzazione e deserializzazione occorre configurare le funzioni serializeUser e deserializeUser: questo può essere fatto all'interno del file principale dell'applicazione o in un file separato, ad esempio auth.js: di seguito è riportato un esempio di configurazione delle funzioni serializeUser e deserializeUser:

```
passport.serializeUser(function(user, done) {
  done(null, user.id);
});

passport.deserializeUser(function(id, done) {
  User.findById(id, function (err, user) {
    done(err, user);
  });
});
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

In questo esempio, la funzione serializeUser restituisce l'ID dell'utente, che viene utilizzato per serializzare i dati utente in modo persistente; la funzione deserializeUser utilizza l'ID utente per cercare l'oggetto utente corrispondente nel database e restituirlo.

Utilizziamo ora Passport per gestire l'autenticazione nelle rotte dell'applicazione; ad esempio, si potrebbe utilizzare la strategia di autenticazione locale per gestire l'accesso all'area protetta dell'applicazione:

```
app.post(
      '/login',
      passport.authenticate('local',
      { successRedirect: '/',failureRedirect: '/login' }));
```

In questo esempio, quando l'utente tenta di accedere all'area protetta dell'applicazione, viene utilizzata la strategia di autenticazione locale per verificare le credenziali dell'utente; se l'autenticazione ha successo, l'utente viene reindirizzato alla pagina principale dell'applicazione, altrimenti viene reindirizzato alla pagina di login.

È ora necessario proteggere le rotte dell'applicazione che richiedono l'autenticazione: per fare questo è possibile utilizzare il middleware di autenticazione di Passport. Ad esempio:

```
app.get(
      '/profile',
      passport.authenticate('local'),
      function(req, res) {
            res.render('profile', { user: req.user });
});
```

In questo esempio, la rotta /profile richiede l'autenticazione dell'utente prima di accedere alla pagina del profilo dell'utente; il middleware di autenticazione di Passport viene utilizzato per verificare l'identità dell'utente prima di consentire l'accesso alla pagina del profilo.

Ricapitolando, per configurare Passport nell'applicazione Node.js, è necessario:

- installare Passport e le eventuali strategie di autenticazione necessarie;
- importare e inizializzare Passport nell'applicazione;
- configurare le strategie di autenticazione necessarie;
- configurare le funzioni serializeUser e deserializeUser;
- utilizzare Passport per gestire l'autenticazione nelle rotte dell'applicazione.

Utilizzando il middleware di autenticazione di Passport, è possibile proteggere le rotte dell'applicazione che richiedono l'autenticazione e creare un'esperienza di autenticazione fluida e sicura per gli utenti dell'applicazione Node.js.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Riportiamo di seguito un esempio di utilizzo di Passport per gestire l'autenticazione utilizzando la strategia di autenticazione locale:

```
const passport = require('passport');
const LocalStrategy = require('passport-local').Strategy;
const User = require('./models/user');

// Configurazione della strategia di autenticazione locale
passport.use(new LocalStrategy(
  function(username, password, done) {
    User.findOne({ username: username }, function (err, user) {
      if (err) { return done(err); }
      if (!user) { return done(null, false); }
      if (!user.verifyPassword(password)) {
           return done(null, false); }
      return done(null, user);
    });
  }
));

// Utilizzo del middleware di autenticazione di Passport per gestire
l'autenticazione nella rotta /login
app.post('/login',
  passport.authenticate('local',
     { successRedirect: '/',
       failureRedirect: '/login' }));

// Utilizzo del middleware di autenticazione di Passport per proteggere
la rotta /profile
app.get('/profile',
  passport.authenticate('local', { failureRedirect: '/login' }),
  function(req, res) {
    res.render('profile', { user: req.user });
  });
```

In questo esempio, la strategia di autenticazione locale viene configurata utilizzando la funzione passport.use(). Questa funzione viene utilizzata per definire come l'autenticazione deve essere gestita per la strategia di autenticazione specificata.

Successivamente, il middleware di autenticazione di Passport viene utilizzato per gestire l'autenticazione nella rotta /login, verificando le credenziali dell'utente utilizzando la strategia di autenticazione locale. Se l'autenticazione ha successo, l'utente viene reindirizzato alla pagina principale dell'applicazione, altrimenti viene reindirizzato alla pagina di login.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Infine, il middleware di autenticazione di Passport viene utilizzato per proteggere la rotta /profile, che richiede l'autenticazione dell'utente prima di accedere alla pagina del profilo dell'utente.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Passport & OAuth

Illustriamo ora un esempio di come utilizzare Passport per gestire l'autenticazione OAuth con Facebook in un'applicazione web.

Occorre installare i pacchetti necessari tramite npm:

```
npm install passport-facebook dotenv
```

Occorre poi creare una nuova applicazione su Facebook Developer e configurare le credenziali dell'applicazione: è necessario configurare correttamente la callback URL dell'applicazione, che Passport utilizzerà per gestire l'autenticazione con Facebook.

Nell'applicazione Node.js, occorre configurare Passport e definire la strategia di autenticazione OAuth con Facebook.

```
const passport = require('passport');
const FacebookStrategy = require('passport-facebook').Strategy;

passport.use(new FacebookStrategy({
    clientID: process.env.FACEBOOK_CLIENT_ID,
    clientSecret: process.env.FACEBOOK_CLIENT_SECRET,
    callbackURL: "/auth/facebook/callback"
  },
  function(accessToken, refreshToken, profile, done) {
    // Verificare l'identità dell'utente e restituirlo
    return done(null, profile);
  }
));
```

In questo esempio, si utilizza la strategia di autenticazione OAuth con Facebook e si specificano le credenziali dell'applicazione Facebook e il callback URL. Inoltre, viene definita la funzione done() che verificherà l'identità dell'utente e restituirà l'oggetto profile.

Infine, è necessario configurare le rotte dell'applicazione per l'autenticazione con Facebook utilizzando Passport.

```
app.get('/auth/facebook', passport.authenticate('facebook'));
app.get('/auth/facebook/callback',
  passport.authenticate('facebook', { failureRedirect: '/login' }),
  function(req, res) {
    // L'autenticazione con Facebook ha successo, reindirizza l'utente
alla pagina principale dell'applicazione
    res.redirect('/');
  });
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

In questo esempio, la prima rotta definisce l'URL per l'avvio dell'autenticazione con Facebook; la seconda rotta gestisce il callback URL specificato in precedenza e utilizza Passport per gestire l'autenticazione con Facebook. Se l'autenticazione ha successo, l'utente viene reindirizzato alla pagina principale dell'applicazione.

Un altro utilizzo comune di Passport con OAuth è l'autenticazione con Google. La configurazione è del tutto simile a quella appena vista per Facebook.

Occorre installare i pacchetti necessari tramite npm:

```
npm install passport-google-oauth20 dotenv
```

Occorre poi creare una nuova applicazione su Google Cloud Console e configurare le credenziali dell'applicazione; anche in questo caso occorre assicurarsi che si sia configurata correttamente la callback URL dell'applicazione, che Passport utilizzerà per gestire l'autenticazione con Google.

Successivamente, occorre configurare Passport nell'applicazione Node.js e definire la strategia di autenticazione OAuth con Google:

```
const passport = require('passport');
const GoogleStrategy=require('passport-google-oauth20').Strategy;
passport.use(new GoogleStrategy({
    clientID: process.env.GOOGLE_CLIENT_ID,
    clientSecret: process.env.GOOGLE_CLIENT_SECRET,
    callbackURL: "/auth/google/callback"
  },
  function(accessToken, refreshToken, profile, done) {
    // Verificare l'identità dell'utente e restituirlo
    return done(null, profile);
  }
));
```

In questo esempio, si utilizza la strategia di autenticazione OAuth con Google e si specificano le credenziali dell'applicazione Google e il callback URL; inoltre, viene definita la funzione done() che verificherà l'identità dell'utente e restituirà l'oggetto profile.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Infine, occorre configurare le rotte dell'applicazione per l'autenticazione con Google utilizzando Passport.

```
app.get('/auth/google',
  passport.authenticate('google', { scope: ['profile'] }));
app.get('/auth/google/callback',
  passport.authenticate('google', { failureRedirect: '/login' }),
  function(req, res) {
    // autenticazione con successo, redirect alla root
    res.redirect('/');
  });
```

In questo esempio, la prima rotta definisce l'URL per l'avvio dell'autenticazione con Google. La seconda rotta gestisce il callback URL specificato in precedenza e utilizza Passport per gestire l'autenticazione con Google. Se l'autenticazione ha successo, l'utente viene reindirizzato alla pagina principale dell'applicazione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- V. Roberto, M. Frailis, A. Gugliotta, P. Omero. Introduzione alle Tecnologie Web, McGraw-Hill, 2010.
- A. S. Tanembaum. Computer Networks (4 ed.). Prentice Hall (4 ed.), 2003.
- Marco Andreini, Patrizia Andronico, Arianna Del Soldato, Cristian Lucchesi, Maurizio Martinelli, Giuseppe Vasarelli. Manuale corso di Tecnologie Web Istituto di Informatica e Telematica del CNR, 2005.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Sitografia

- HTML 4.01 Specification: http://www.w3.org/TR/html401/
- HTML5: http://www.w3.org/TR/html5/
- Javascript: JavaScript Tutorial (w3schools.com)
- HTML: http://www.w3schools.com/html/default.asp
- CSS: https://www.w3schools.com/css/default.asp
- CSS: http://www.w3.org/TR/CSS/
- NODEJS: https://nodejs.org

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).